claude-windows / ac48091f-a4f4-4268-b968-75205d9a50ff

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\subagents\workflows\wf_b8ec0894-3cb\agent-aa857084ec4f99981.jsonl`
- SHA-256: `d54dd70fa8b0ae2bf0d48a1a95d77209959f5f241ebf373e410e8acce776ea59`
- Source modified: `2026-06-21T19:14:39+00:00`
- Imported at: `2026-07-08T15:59:55+00:00`
- project: `wf_b8ec0894-3cb`
- session_id: `ac48091f-a4f4-4268-b968-75205d9a50ff`

Transcript

1. user (2026-06-21T19:12:53.046Z)

```
You are verifying ONE claim in khelsutra's deploy runbook (deploy/DEPLOY_ALPHA.md) against the
REAL code the owner will deploy.

CRITICAL repo rules:
- Engine repo is at /c/Users/avidu/Projects/badminton-highlight-indexer. Its working tree is
checked out on a STALE concurrent-session branch (12 commits behind master). DO NOT read the
working tree for engine claims and DO NOT checkout/commit/modify anything there ? it is a shared
checkout owned by another session.
- To read what the owner actually deploys, read ENGINE origin/master ONLY, via:
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer grep -n "PATTERN" origin/master
-- 'PATHGLOB'      (search master's tree)
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer show origin/master:PATH
(read a master file)
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer log origin/master --oneline -i
--grep="..."        (history)
  (I already ran 'git fetch' so origin/master is current. Never fetch/checkout.)
- khelsutra repo is at /c/Users/avidu/Projects/khelsutra, working tree is on clean master ? you
MAY use Read/Grep/Glob directly there.

Be adversarial and literal: confirm the claim is TRUE for origin/master, with file:line
evidence. If the doc says X but the code says Y, that is a 'discrepancy' ? capture it precisely.
If something the owner must DO (beyond the doc's text) is implied, put it in owner_action_note.
Verdict 'confirmed' only with concrete evidence; 'partial' if mostly true with a caveat;
'refuted' if wrong; 'unknown' if you genuinely could not determine it.

----
CLAIM (§3 + §7): the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64)
to target the api. origin, defaulting to same-origin `/api`; Pages build is root=web, command
`npm run build`, output `web/dist`; and browser upload chunks large files to ?90MB (the §7 100MB
Cloudflare cap), reel downloads cross-origin via Content-Disposition.
This is the khelsutra repo at /c/Users/avidu/Projects/khelsutra (working tree on clean master ?
use Read/Grep directly). Verify: (a) the SPA reads VITE_API_BASE and falls back to `/api` when
unset (find the api client); (b) web/package.json has a `build` script and Vite outputs to dist;
(c) the chunked upload path (hooks/useUpload, api/client) and its chunk size ? confirm it's
?90MB or report the actual size; (d) where the reel download relies on Content-Disposition.
Report exact file:line.
```

2. assistant (2026-06-21T19:12:55.643Z)

I'll verify this claim against the khelsutra repo. Let me start by pulling the latest and finding the relevant files.

3. user (2026-06-21T19:12:58.954Z)

```
Already up to date.
```

4. user (2026-06-21T19:12:59.194Z)

```
deploy\DEPLOY_ALPHA.md:14:    ??? app.khelsutra.guru  ? Cloudflare Pages         (the web/ SPA;
built with VITE_API_BASE)
deploy\DEPLOY_ALPHA.md:23:- **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are
different origins ? the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR
#64) and the engine locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code
change ? it's already env-driven).
deploy\DEPLOY_ALPHA.md:84:   - **Environment variable:** `VITE_API_BASE =
https://api.khelsutra.guru/api`  ? this is what points the SPA at the engine tunnel (PR #64).
deploy\DEPLOY_ALPHA.md:141:- 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared
+ Pages + CF Access) consuming the engine exposure-safety boot posture (engine #294) + the SPA
`VITE_API_BASE` (#64). Pairs with `LOCAL_APPLIANCE_ARCHITECTURE.md` (single-origin alternative)
+ the engine `HOSTING_PLAN.md` §2.
web\src\vite-env.d.ts:9:  readonly VITE_API_BASE?: string;
web\src\api\client.ts:19: *  set `VITE_API_BASE` at build time to the engine's API base
**including the `/api` path**
web\src\api\client.ts:24:  return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
web\src\api\client.test.ts:27:  it("defaults to same-origin /api when VITE_API_BASE is unset",
() => {
web\src\api\client.test.ts:31:  it("uses VITE_API_BASE for a split deploy (SPA on Pages + api.*
tunnel)", () => {
web\src\api\client.test.ts:32:    vi.stubEnv("VITE_API_BASE", "https://api.khelsutra.guru/api");
web\src\api\client.test.ts:37:    vi.stubEnv("VITE_API_BASE",
"https://api.khelsutra.guru/api/");
docs\LOCAL_APPLIANCE_ARCHITECTURE.md:175:[Omitted long matching line]
```

5. assistant (2026-06-21T19:13:02.493Z)

```
Found the API client. Let me read the relevant files: the api client, the upload hook/path,
package.json, and the DEPLOY_ALPHA.md sections.
```

6. user (2026-06-21T19:13:03.449Z)

```
1       // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2       // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3       // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4       import { log, newCorrelationId } from "../lib/log";
5       import type {
6         RallySegment,
7         QueryResponse,
8         CompileResponse,
9         EngineConfig,
10        ProcessResponse,
11        JobStatus,
12        UploadInitResponse,
13        UploadPartResponse,
14        UploadCompleteResponse,
15      } from "./types";
16
17      /** Resolve the engine API base. Default same-origin `"/api"`. For a SPLIT deploy ? SPA
on
18       *  Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md §2) ?
19       *  set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
20       *  (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
21       *  Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
```

```
so
22       *  `` `${base}/process` `` never double-slashes. */
23      export function apiBase(): string {
24        return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
25      }
26
27      const BASE = apiBase();
28
29      export class ApiError extends Error {
30        readonly cid?: string;
31        readonly status?: number;
32        readonly body?: string;
33        constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
34          super(message);
35          this.name = "ApiError";
36          this.cid = opts.cid;
37          this.status = opts.status;
38          this.body = opts.body;
39          if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
40        }
41      }
42
43      interface RequestOpts {
44        method?: string;
45        body?: string;
46        headers?: Record<string, string>;
47        /** Correlation id to thread through this call; auto-generated if absent. */
48        cid?: string;
49      }
50
51      async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
52        const cid = opts.cid ?? newCorrelationId();
53        const url = `${BASE}${path}`;
54        const method = opts.method ?? "GET";
55        log("info", "api.request", { cid, method, url });
56
57        let res: Response;
58        try {
59          res = await fetch(url, {
60            method,
61            body: opts.body,
62            headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
63          });
64        } catch (err) {
65          // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
66          log("error", "api.network_error", { cid, method, url, error: String(err) });
67          throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
68        }
69
70        if (!res.ok) {
71          const body = await res.text().catch(() => "");
72          log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
73          throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
74        }
75
76        const data = (await res.json()) as T;
77        log("info", "api.response", { cid, method, url, status: res.status });
78        return data;
79      }
80
81      /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
```

```
for
82       *  processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
83       *  (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
84      export async function processVideo(videoPath: string, cid?: string, locale?: string):
Promise<ProcessResponse> {
85        // `locale` (the UI language) is recorded for PMF analytics (which markets use the
tool). Optional
86        // + omitted when absent (JSON.stringify drops undefined); the engine ignores it until
wired.
87        return request<ProcessResponse>("/process", {
88          method: "POST",
89          body: JSON.stringify({ video_path: videoPath, locale }),
90          cid,
91        });
92      }
93
94      /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
95      export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
96        return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
97      }
98
99      /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
100     export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
101       const data = await request<QueryResponse>("/query", {
102         method: "POST",
103         body: JSON.stringify({ video_id: videoId }),
104         cid,
105       });
106       return data.play_segments ?? [];
107     }
108
109     /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
110     export async function compileReel(
111       videoId: string,
112       segmentIds: number[],
113       outputFilename: string,
114       cid?: string,
115       locale?: string,
116     ): Promise<CompileResponse> {
117       // `locale` (the UI language) localizes the reel's branding watermark engine-side.
Optional +
118       // omitted when absent; the engine falls back to the default watermark for an
unknown/absent locale.
119       return request<CompileResponse>("/compile", {
120         method: "POST",
121         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
122         cid,
123       });
124     }
125
126     /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
127     export function highlightUrl(videoId: string, filename: string): string {
128       return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
129     }
130
131     /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
```

```
    export async function getConfig(cid?: string): Promise<EngineConfig> {
      return request<EngineConfig>("/config", { method: "GET", cid });
    }

    /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
    export function minShotCount(cfg: EngineConfig): number | undefined {
      return cfg.stitching?.min_shot_count;
    }

    // ?? chunked upload (backend/api/uploads.py) ????????????????????????????????
    // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
    // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.

    /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
    export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
      return request<UploadInitResponse>("/upload/init", {
        method: "POST",
        body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
        cid,
      });
    }

    /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
    export async function uploadPart(
      uploadId: string,
      index: number,
      chunk: Blob,
      cid?: string,
      signal?: AbortSignal,
    ): Promise<UploadPartResponse> {
      const c = cid ?? newCorrelationId();
      const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
      log("info", "api.request", { cid: c, method: "PUT", url });
      let res: Response;
      try {
        res = await fetch(url, {
          method: "PUT",
          body: chunk,
          headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
          signal,
        });
      } catch (err) {
        log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
});
        throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
      }
      if (!res.ok) {
        const body = await res.text().catch(() => "");
        log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
        throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
      }
      log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
      return (await res.json()) as UploadPartResponse;
    }

    /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
*/
```

```
    export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
      return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
        method: "POST",
        body: JSON.stringify({ total_parts: totalParts }),
        cid,
      });
    }

    /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[uploads.py:147]. */
    export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
      await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
    }

    export interface UploadProgress {
      sentBytes: number;
      totalBytes: number;
    }

    /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
     *  On ANY failure or abort it best-effort DELETEs the server-side partial, then
rethrows so the
     *  caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}.
     *  Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
recovery is a
     *  fresh init (re-pick the file), which the UI's retry does. */
    export async function uploadFile(
      file: File,
      opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {},
    ): Promise<UploadCompleteResponse> {
      const cid = opts.cid ?? newCorrelationId();
      const init = await uploadInit(file.name, file.size, cid);
      // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
      // would silently upload zero parts) ? fall back to the engine's 95 MB default.
      const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
      const partBytes = partMb * 1024 * 1024;
      const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
      log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
      let sent = 0;
      try {
        for (let i = 0; i < totalParts; i++) {
          if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
          const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
          await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
          sent += chunk.size;
          opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
        }
        const done = await uploadComplete(init.upload_id, totalParts, cid);
        log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
        return done;
      } catch (err) {
        log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
        await uploadAbort(init.upload_id, cid).catch(() => {});
        throw err;
```

```
238       }
239     }
240
```

7. user (2026-06-21T19:13:03.935Z)

```
1       {
2         "name": "khelsutra-web",
3         "private": true,
4         "version": "0.0.0",
5         "type": "module",
6         "description": "KhelSutra Studio ? operator console SPA (drives the rally indexer
engine).",
7         "scripts": {
8           "dev": "vite",
9           "build": "vite build",
10          "preview": "vite preview",
11          "typecheck": "tsc --noEmit",
12          "test": "vitest run"
13        },
14        "dependencies": {
15          "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
16          "@fontsource-variable/noto-sans-kannada": "^5.2.9",
17          "i18next": "^26.3.1",
18          "i18next-browser-languagedetector": "^8.2.1",
19          "i18next-icu": "^2.4.3",
20          "react": "^19.0.0",
21          "react-dom": "^19.0.0",
22          "react-i18next": "^17.0.8"
23        },
24        "devDependencies": {
25          "@tailwindcss/vite": "^4.0.0",
26          "@testing-library/dom": "^10.4.1",
27          "@testing-library/react": "^16.3.2",
28          "@types/node": "^26.0.0",
29          "@types/react": "^19.0.0",
30          "@types/react-dom": "^19.0.0",
31          "@vitejs/plugin-react": "^4.3.4",
32          "jsdom": "^29.1.1",
33          "tailwindcss": "^4.0.0",
34          "typescript": "^5.7.0",
35          "vite": "^6.0.0",
36          "vitest": "^3.0.0"
37        }
38      }
39
```

8. user (2026-06-21T19:13:04.377Z)

```
1       # Deploy KhelSutra Studio for alpha ? a shareable page, no SSH
2
3       > **Status:** `READY` (owner executes the cloud steps) · **Topology:** split ? SPA on
Cloudflare Pages + engine on an `api.*` cloudflared tunnel, Cloudflare Access over both (the
engine's `HOSTING_PLAN.md` §2 alpha plan). · **Cost:** ~$0/mo standing (Pages + Tunnel + Access
are free ?50 users) + Gemini per-run (~$0.05/match).
4
5       This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a
**page you can share with alpha testers** ? they open `app.khelsutra.guru`, sign in (email
allowlist), point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.
6
7       ## 0. The shape
8
9       ```
10       tester browser
11          ? https
```

```
         ?
      Cloudflare edge (free): DNS · TLS · Access (email-OTP allowlist) in front of BOTH
hostnames
         ??? app.khelsutra.guru  ? Cloudflare Pages         (the web/ SPA; built with
VITE_API_BASE)
         ??? api.khelsutra.guru  ? cloudflared Tunnel (outbound-only) ?? 127.0.0.1:8000 on
YOUR box
                                                                          ?  FastAPI engine
                                                                          ?  (Gemini
segmenter; GPU optional)
                                                                          ?  async jobs,
SQLite, output/ on disk
```

- **No inbound ports.** `cloudflared` dials out; the engine stays bound to `127.0.0.1:8000`. The box's home/public IP is never exposed.
- **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also* verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a direct hit to the tunnel can't spoof identity.
- **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ? the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's already env-driven).

> **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA *and* the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see `LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose (stable Pages-hosted page + a thin API tunnel).

---

1. Prerequisites

- A **box** to run the engine: the CPU droplet `168.144.126.176` (Gemini-only ? no GPU; fine for alpha) **or** your Windows+WSL GPU box (real WASB). `ffmpeg` + Python 3.8+ installed; the `badminton-highlight-indexer` engine checked out.
- A **Cloudflare account** with the `khelsutra.guru` zone (you already run the marketing site there).
- `cloudflared` installed on the box (`https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/downloads/`).
- A **fresh** `GEMINI_API_KEY` (rotate the chat-shared one ? it uploads downsized chunks to Google; this is a paid + cloud action).

---

2. Engine (the `api.` side)

2.1 Install (with the auth extra)

```bash
cd badminton-highlight-indexer
pip install -e .[auth]          # PyJWT[crypto] ? REQUIRED when edge auth is on, else every request 500s
```

2.2 The safe-exposed config

Create / edit `config.json` on the box:

```jsonc
{
  "security": {
    "edge_auth_enabled": true,                       // turn the CF-Access JWT verify ON
    "allowed_video_root": "/srv/khelsutra/incoming",  // the path-jail ? REQUIRED when exposed
    "upload_dir": "/srv/khelsutra/incoming/uploads",  // MUST sit UNDER allowed_video_root (see §7)
    "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
```

```
        "cf_access_aud": "<the CF Access application AUD>"  // filled in §4
      }
    }
    ```
    And export the env (a `.env` or your shell):
    ```bash
    export GEMINI_API_KEY="<your-rotated-key>"
    export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru"   # lock CORS to the SPA origin
    ```

    > **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails fast** if `edge_auth_enabled` is on but the `allowed_video_root` jail or `cf_team_domain`/`cf_access_aud` is missing, and **warns** if `PyJWT` isn't installed. So a half-configured exposure can't silently come up ? fix what it prints and restart.

    ### 2.3 Run
    ```bash
    python -m backend.main --server     # binds 127.0.0.1:8000 (the safe default ? do NOT change the bind)
    ```
    Keep it alive with a process manager (`systemd` on the droplet, mirroring `site/scripts/provision-vm.sh`'s unit; or `pm2`/`nssm` on Windows).

    ## 3. The SPA on Cloudflare Pages (the `app.` side)

    The SPA ships as a **static Pages site**, git-connected like the marketing site (auto-deploy on merge to `master`).

    1. **Cloudflare dashboard ? Workers & Pages ? Create ? Pages ? Connect to Git** ? pick the `khelsutra` repo.
    2. **Build settings:**
       - **Root directory:** `web`
       - **Build command:** `npm run build`
       - **Build output directory:** `web/dist`
       - **Environment variable:** `VITE_API_BASE = https://api.khelsutra.guru/api` ? this is what points the SPA at the engine tunnel (PR #64).
    3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.

    > Every push to `master` now rebuilds + redeploys the SPA. To preview a change first, Pages builds a per-PR preview URL.

    ## 4. Cloudflare Access (gate BOTH hostnames)

    1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
    2. **Application domain:** add **both** `app.khelsutra.guru` **and** `api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester signs in once and both the SPA and its API calls are authorized).
    3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50 users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
    4. **Copy the application `AUD` tag** ? paste into the engine's `config.json` `security.cf_access_aud` (§2.2), and set `cf_team_domain` to `https://<your-team>.cloudflareaccess.com`. Restart the engine.
    5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to `api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass options requests to CORS preflight?** (or add an explicit OPTIONS bypass), else the preflight is redirected to the login page and every API call fails. The engine's CORS middleware (`allow_credentials=false`, no cookies) then answers the preflight.

```
```

5. The tunnel (engine ? Cloudflare)

```bash
cloudflared tunnel login                                    # browser auth to your zone
cloudflared tunnel create khelsutra-studio                  # note the <TUNNEL_UUID>
# copy deploy/cloudflared.config.example.yml ? /etc/cloudflared/config.yml, fill <TUNNEL_UUID>
cloudflared tunnel route dns khelsutra-studio api.khelsutra.guru
cloudflared tunnel run khelsutra-studio                     # or install as a service: `cloudflared service install`
```

See `deploy/cloudflared.config.example.yml` for the ingress (only `api.khelsutra.guru ? http://127.0.0.1:8000`).

---

6. Verify (the posture test + the CUJ)

Exposure posture (must hold ? this is what makes it safe to share):
- From OUTSIDE the box: `curl --max-time 5 http://<box-public-ip>:8000/api/health` ? **refused / timeout** (the engine is `127.0.0.1`-bound; no inbound port). ? The box is not directly reachable.
- `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to the Access login** (not a 200). ? The gate is in front.
- Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the SPA loads and `/api/health` returns 200 (same session cookie). ?

The shareable CUJ (what a tester does): open `app.khelsutra.guru` ? sign in ? paste a `gs://?` video (or a path under `allowed_video_root`) ? **Process** ? watch progress ? pick rallies ? **Build** ? **download the reel**. All from the page.

---

7. Honest caveats / deferred

- **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole). Keep `upload_dir` inside the jail (§2.2).
- **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app JWT verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
- **Cross-origin reel download:** the reel streams from `api.` with `Content-Disposition: attachment`, so it downloads even cross-origin ? but verify the filename in your browser during the CUJ (cross-origin `<a download>` filename hints are ignored; the header is the authority).
- **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large matches upload as ?90 MB chunks (the SPA chunked-upload path), or pre-stage to a bucket and ingest the `gs://` URI (already supported).
- **Box offline = engine offline.** The Pages SPA stays up and shows a friendly "engine offline" banner (`/api/health` ping); processing resumes when the box is back. Beta removes this with a queue (`HOSTING_PLAN.md` §3).
- **Gemini = paid + uploads footage to Google.** Keep the key rotated; use it only for clips you're OK sending.

---

8. Teardown

`cloudflared tunnel cleanup khelsutra-studio` + delete the tunnel; remove the Pages custom domain; disable the Access application; stop the engine service. The box's `output/` + `uploads/` retain originals ? sweep them per your retention policy before any wider exposure.

Changelog

- 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared + Pages + CF Access) consuming the engine exposure-safety boot posture (engine #294) + the SPA `VITE_API_BASE` (#64). Pairs with `LOCAL_APPLIANCE_ARCHITECTURE.md` (single-origin alternative)

```
+ the engine `HOSTING_PLAN.md` §2.
142
```

9. assistant (2026-06-21T19:13:09.840Z)

```
I have most of the picture. Key finding so far: the doc (§7 line 130) claims chunks are "?90 MB"
but the SPA client honors the engine's `max_part_mb` from `/upload/init`, with a fallback
default of **95 MB** (client.ts:218), not 90 MB. Let me check the upload hook, the Vite config
(output dir + chunk size source), and the reel download's reliance on Content-Disposition.
```

10. user (2026-06-21T19:13:10.577Z)

```
web\src\hooks\useUpload.test.ts
web\src\hooks\useUpload.ts
```

11. user (2026-06-21T19:13:11.275Z)

```
1       import { defineConfig } from "vitest/config";
2       import react from "@vitejs/plugin-react";
3       import tailwindcss from "@tailwindcss/vite";
4
5       // The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
6       // proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
7       // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local
engine
8       // (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if
needed.
9       export default defineConfig({
10        plugins: [react(), tailwindcss()],
11        server: {
12          proxy: {
13            "/api": { target: "http://localhost:8000", changeOrigin: true },
14          },
15        },
16        test: {
17          // Default env is node (the pure lib/parser tests); the i18n render test opts into
jsdom via a
18          // `// @vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
19          environment: "node",
20          include: ["src/**/*.test.{ts,tsx}"],
21          // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files
no-op.
22          setupFiles: ["src/test/setup.ts"],
23        },
24      });
25
```

12. user (2026-06-21T19:13:12.407Z)

```
web\src\api\types.ts:35:  /** server-local path (NOT a URL) ? build the download URL with
highlightUrl(). */
web\src\api\client.ts:126:/** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
web\src\api\client.ts:127:export function highlightUrl(videoId: string, filename: string):
string {
web\src\api\client.test.ts:2:import { apiBase, listRallies, compileReel, highlightUrl,
getConfig, minShotCount, ApiError } from "./client";
web\src\api\client.test.ts:42:describe("highlightUrl", () => {
web\src\api\client.test.ts:44:    expect(highlightUrl("abc123", "my
reel.mp4")).toBe("/api/highlights/abc123/my%20reel.mp4");
web\src\App.tsx:17:import { ApiError, compileReel, getConfig, highlightUrl, listRallies,
minShotCount } from "./api/client";
web\src\App.tsx:122:  // Build ? POST /api/compile ? preview/download the stitched reel. One cid
```

```
traces the whole call.
web\src\App.tsx:148:      const url = highlightUrl(videoId, filename);
web\src\components\CompileResult.tsx:2:// preview the reel inline (<video> over GET
/api/highlights/...) and offer a download. The engine
web\src\components\CompileResult.tsx:70:            download={build.filename}
web\src\components\CompileResult.tsx:73:            {t("result.download", { filename:
build.filename })}
```

13. assistant (2026-06-21T19:13:14.742Z)

Let me read the upload hook and the CompileResult download component to confirm the chunk-size source and the Content-Disposition reliance.

14. user (2026-06-21T19:13:15.697Z)

```
1       // Drives the in-UI file UPLOAD half of ingest (tracker T4): chunk a local file up to
the engine's
2       // /api/upload/* endpoints, report byte progress, then hand the returned server PATH to
the existing
3       // processing back-half (useIngestJob.submit). Alpha users have a FILE, not a gs:// URI
? this is the
4       // keystone that lets them point the tool at it from the browser.
5       //
6       // Kept i18n-FREE (returns stable IngestErrorCodes via classifyError, not prose) so it
unit-tests
7       // without a translation context ? IngestPanel maps codes to localized strings via
errorText. Abort
8       // is an AbortController: cancel/unmount/a superseding pick stops the in-flight upload
and the
9       // orchestrator best-effort DELETEs the server-side partial.
10      import { useCallback, useEffect, useRef, useState } from "react";
11      import { uploadFile } from "../api/client";
12      import { classifyError, type IngestError } from "../lib/errors";
13      import { log, newCorrelationId } from "../lib/log";
14
15      export type UploadPhase =
16        | { status: "idle" }
17        | { status: "uploading"; pct: number; filename: string }
18        | { status: "error"; error: IngestError };
19
20      export function useUpload(onComplete: (path: string) => void) {
21        const [phase, setPhase] = useState<UploadPhase>({ status: "idle" });
22        const ctrlRef = useRef<AbortController | null>(null);
23        // Keep the latest onComplete without making start depend on it (a parent re-render
won't restart).
24        const onCompleteRef = useRef(onComplete);
25        onCompleteRef.current = onComplete;
26
27        const stop = useCallback(() => {
28          ctrlRef.current?.abort();
29          ctrlRef.current = null;
30        }, []);
31
32        // Abort any in-flight upload on unmount.
33        useEffect(() => stop, [stop]);
34
35        const start = useCallback(
36          (file: File) => {
37            // A 0-byte "video" is never valid ? reject before touching the network (reuse
invalidUri copy).
38            if (!file || file.size === 0) {
39              setPhase({ status: "error", error: { code: "invalidUri" } });
40              return;
41            }
42
```

```
43            stop(); // supersede any prior upload
44            const ctrl = new AbortController();
45            ctrlRef.current = ctrl;
46            const cid = newCorrelationId();
47            setPhase({ status: "uploading", pct: 0, filename: file.name });
48            log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });
49
50            void (async () => {
51              try {
52                const res = await uploadFile(file, {
53                  cid,
54                  signal: ctrl.signal,
55                  onProgress: ({ sentBytes, totalBytes }) => {
56                    if (ctrl.signal.aborted) return;
57                    const pct = totalBytes > 0 ? Math.min(100, Math.round((sentBytes /
totalBytes) * 100)) : 0;
58                    setPhase({ status: "uploading", pct, filename: file.name });
59                  },
60                });
61                if (ctrl.signal.aborted) return;
62                setPhase({ status: "idle" }); // hand off ? the engine processing phase takes
over
63                onCompleteRef.current(res.path);
64              } catch (err) {
65                // User-cancelled (abort) is silent, not an error.
66                if (ctrl.signal.aborted || (err instanceof DOMException && err.name ===
"AbortError")) return;
67                setPhase({ status: "error", error: classifyError(err) });
68                log("error", "upload.failed", { cid, filename: file.name, error: String(err)
});
69              }
70            })();
71          },
72          [stop],
73        );
74
75        const cancel = useCallback(() => {
76          stop();
77          setPhase({ status: "idle" });
78          log("info", "upload.cancel", {});
79        }, [stop]);
80
81        const reset = useCallback(() => {
82          stop();
83          setPhase({ status: "idle" });
84        }, [stop]);
85
86        const busy = phase.status === "uploading";
87        return { phase, start, cancel, reset, busy };
88      }
89
```

15. user (2026-06-21T19:13:16.228Z)

```
55              </button>
56            </div>
57
58            {build.status === "done" ? (
59              <div className="mt-3">
60                {/* eslint-disable-next-line jsx-a11y/media-has-caption ? a stitched rally
reel has no captions */}
61                <video
62                  data-testid="reel-video"
63                  src={build.url}
64                  controls
```

```
65                  playsInline
66                  className="w-full max-w-xl rounded-lg bg-black"
67                />
68                <a
69                  href={build.url}
70                  download={build.filename}
71                  className="mt-3 inline-flex min-h-11 items-center rounded-full bg-green px-4
py-2 text-sm font-semibold text-[#04140D]"
72                >
73                  {t("result.download", { filename: build.filename })}
74                </a>
75              </div>
76            ) : (
77              <p className="mt-2 text-sm text-red-800" data-testid="compile-error">
78                {build.error}
79              </p>
80            )}
81          </section>
82        );
83      }
84
```

16. assistant (2026-06-21T19:13:19.744Z)

Now I have the full picture. Let me confirm the engine's actual `max_part_mb` default on origin/master, since the doc says "?90 MB" but the SPA derives the chunk size from the engine. This determines whether the doc's "?90 MB" claim is accurate.

17. user (2026-06-21T19:13:21.676Z)

<persisted-output>
Output too large (63KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\tool-results\bjxfl0e60.txt

Preview (first 2KB):
origin/master:backend/api/uploads.py:3:Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
origin/master:backend/api/uploads.py:49:   part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
origin/master:backend/api/uploads.py:85:   return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
origin/master:backend/config/models.py:293:   max_part_mb: int = 95
origin/master:backend/ui/assets/index-ZjpoX6Fg.js:60:In order to be iterable, non-array objects must have a [Symbol.iterator]() method.`)}function E0(u,l){if(u){if(typeof u=="string")return Sg(u,l);var s=Object.prototype.toString.call(u).slice(8,-1);if(s==="Object"&&u.constructor&&(s=u.constructor.name),s==="Map"||s==="Set")return Array.from(u);if(s==="Arguments"||/^(?:Ui|I)nt(?:8|16|32)(?:Clamped)?Array$/.test(s))return Sg(u,l)}}function Sg(u,l){(l==null||l>u.length)&&(l=u.length);for(var s=0,r=new Array(l);s<l;s++)r[s]=u[s];return r}function T0(u,l){if(!(typeof Symbol>"u"||!(Symbol.iterator in Object(u)))){var s=[],r=!0,c=!1,f=void 0;try{for(var g=u[Symbol.iterator](),d;!(r=(d=g.next()).done)&&(s.push(d.value),!(l&&s.length===l));r=!0);}catch(p){c=!0,f=p}finally{try{!r&&g.return!=null&&g.return()}finally{if(c)throw f}}return s}}function x0(u){if(Array.isArray(u))return u}function vu(u){"@babel/helpers - typeof";return typeof Symbol=="function"&&typeof Symbol.iterator=="symbol"?vu=function(s){return typeof s}:vu=function(s){return s&&typeof Symbol=="function"&&s.constructor===Symbol&&s!==Symbol.prototype?"symbol":typeof s},vu(u)}function Eg(u,l){var s=Object.keys(u);if(Object.getOwnPropertySymbols){var r=Object.getOwnPropertySymbols(u);l&&(r=r.filter(function(c){return Object.getOwnPropertyDescriptor(u,c).enumerable})),s.push.apply(s,r)}return s}function Tg(u){for(var l=1;l<arguments.length;l++){var s=arguments[l]!=null?arguments[l]:{};l%2?Eg(Object(s),!0).forEach(function(r){A0(u,r,s[r])}):Object.getOwnPropertyDescriptors?Object.defineProperties(u,
...
</persisted-output>

18. assistant (2026-06-21T19:13:26.075Z)

Confirmed. The engine's `security.max_part_mb` default is **95** (origin/master `backend/config/models.py:293` and `backend/api/uploads.py:49`). Let me verify whether the engine also enforces a `/api/process` Content-Disposition on the highlights download and check the engine's highlights endpoint for the Content-Disposition header (the doc §7 line 129 claims the reel streams with `Content-Disposition: attachment`).

19. user (2026-06-21T19:13:27.630Z)

(Bash completed with no output)

20. assistant (2026-06-21T19:13:30.965Z)

No match in those files. Let me search the engine more broadly for how the highlights endpoint serves the reel and whether it sets Content-Disposition.

21. user (2026-06-21T19:13:32.714Z)

```
origin/master:backend/config/models.py:231:    default_highlights_name: str = "highlights.mp4"
origin/master:backend/pipeline/stitcher/compiler.py:204:        per-segment body of
compile_highlights). Returns (clip_path, skip_record): on
origin/master:backend/pipeline/stitcher/compiler.py:305:    def compile_highlights(self,
raw_video_path: str, segments: List[Tuple[float, float]], output_filename: str) -> str:
origin/master:backend/pipeline/stitcher/compiler.py:308:        into a seamless highlights file,
applying end_padding to prevent abrupt endings.
origin/master:backend/reporting/harvest.py:162:def _highlights(rec: "obsreport.RunRecorder",
play_segments: List[Dict[str, Any]],
origin/master:backend/reporting/harvest.py:171:        notes.append("No highlights were produced
for this video.")
origin/master:backend/reporting/harvest.py:172:
rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
origin/master:backend/reporting/harvest.py:220:    _highlights(rec, play_segments, vm)
origin/master:backend/tools/export.py:156:def segments_to_edl(segments: Iterable[Dict[str,
Any]], title: str = "Highlights") -> str:
```

22. assistant (2026-06-21T19:13:35.091Z)

Let me find the actual route that serves the highlight file (GET /api/highlights/...) on the engine to confirm how the download is served.

23. user (2026-06-21T19:13:36.776Z)

```
origin/master:backend/main.py:38:from fastapi.responses import FileResponse, HTMLResponse,
JSONResponse
origin/master:backend/main.py:421:    Mirrors /api/highlights' safety (bare-name + containment
checks)."""
origin/master:backend/main.py:435:    return FileResponse(path, media_type=media, filename=name)
origin/master:backend/main.py:669:@app.get("/api/highlights/{video_id}/{filename}")
origin/master:backend/main.py:690:    return FileResponse(path, media_type=media, filename=name)
origin/master:backend/main.py:767:        download_url =
f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
origin/master:backend/main.py:863:    parser.add_argument("--output-dir", default="output",
help="Directory where processed clips/highlights are saved")
```

24. assistant (2026-06-21T19:13:40.077Z)

The engine serves the highlight via `FileResponse(path, media_type=media, filename=name)` at `backend/main.py:690`. FastAPI's `FileResponse` with a `filename` argument automatically sets `Content-Disposition: attachment; filename="..."`. This confirms the doc's §7 line 129 claim that the reel streams with `Content-Disposition: attachment`. Let me read that handler to be precise.

25. user (2026-06-21T19:13:41.129Z)

```
@app.get("/api/highlights/{video_id}/{filename}")
def download_highlight(video_id: str, filename: str):
    """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
    (which only writes into output_dir; the browser previously got back an unreachable
    server-local path)."""
    try:
        name = safe_output_filename(filename)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    if not db_instance.get_video(video_id):
        raise HTTPException(status_code=404, detail="Indexed video record not found.")
    output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
    path = os.path.join(output_dir, name)
    try:
        path = resolve_under_root(path, output_dir)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    if not os.path.isfile(path):
        raise HTTPException(status_code=404,
                            detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
    media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
    return FileResponse(path, media_type=media, filename=name)
```

26. assistant (2026-06-21T19:13:56.028Z)

All confirmed. Starlette's `FileResponse` with a `filename` kwarg sets `Content-Disposition: attachment; filename="..."` automatically. Now I have everything needed.

Summary of findings against the claim:

(a) VITE_API_BASE + fallback ? CONFIRMED. `web/src/api/client.ts:23-24`: `apiBase()` returns `(import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "")` ? reads the var, falls back to same-origin `/api`, trims trailing slash. Type declared at `web/src/vite-env.d.ts:9`. Tests at `web/src/api/client.test.ts:27-37` assert both the `/api` default and the `https://api.khelsutra.guru/api` split value (and trailing-slash trim).

(b) Pages build (root=web, `npm run build`, output `web/dist`) ? CONFIRMED. `web/package.json:9` has `"build": "vite build"`. Vite's default output is `dist`, relative to root=`web` ? `web/dist` (no `build.outDir` override in `web/vite.config.ts`, which would otherwise change it). The doc §3 build settings (root `web`, command `npm run build`, output `web/dist`) match.

(c) Chunk size ?90 MB ? REFUTED / discrepancy. The doc §7 line 130 says large matches "upload as ?90 MB chunks." The SPA does NOT hardcode 90 MB: `client.ts:218-219` derives `partMb` from the engine's `/upload/init` response `max_part_mb`, with a fallback of **95** when the value is missing/non-finite. The engine's default `max_part_mb` is **95** (origin/master `backend/config/models.py:293` = `95`; `backend/api/uploads.py:49` `getattr(sec, "max_part_mb", 95)`). So with default config the chunks are **95 MB**, not ?90 MB ? and 95 MB is still under Cloudflare's 100 MB cap, so it works, but the doc's specific "?90 MB" number is wrong. The upload path is `useUpload.ts` ? `uploadFile()` (`client.ts:210-239`) ? sequential `uploadPart` PUTs (`client.ts:155-184`).

(d) Reel download relies on Content-Disposition ? CONFIRMED. SPA renders `<a href={build.url} download={build.filename}>` (`CompileResult.tsx:68-74`) where `build.url` is `highlightUrl()` (`client.ts:127-129`) pointing at the cross-origin `api.` host. Because cross-origin `<a download>` filename hints are ignored, the header is authoritative ? and the engine serves the reel via `FileResponse(path, media_type=media, filename=name)` at origin/master `backend/main.py:690`, which Starlette emits as `Content-Disposition: attachment; filename="..."`. Doc §7 line 129 correctly notes the header is the authority.

27. user (2026-06-21T19:14:12.500Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be

parsed as JSON.
You sent (first 200 of 3521 bytes): {"claim": "The SPA is built with
VITE_API_BASE=https://api.khelsutra.guru/api (PR #64) defaulting to same-origin /api; Pages
build is root=web, `npm run build`, output web/dist; browser chunks large u
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters,
or truncated output. Retry with valid JSON.</tool_use_error>

28. assistant (2026-06-21T19:14:14.357Z)

I'll resend with proper JSON structure.

29. user (2026-06-21T19:14:30.529Z)

Structured output provided successfully

30. assistant (2026-06-21T19:14:39.187Z)

Verified the claim against khelsutra master (SPA) and the engine origin/master.

Verdict: **partial** ? three of the four sub-claims are confirmed exactly; the chunk-size figure
is off.

- (a) VITE_API_BASE + `/api` fallback ? CONFIRMED at `web/src/api/client.ts:23-24`, typed at
`vite-env.d.ts:9`, tested at `client.test.ts:27-37`.
- (b) Pages build root=web / `npm run build` / output `web/dist` ? CONFIRMED
(`web/package.json:9`; Vite default `dist` under root=web, no `outDir` override).
- (c) Chunk size ? REFUTED on the number: the doc says "?90 MB" but the SPA derives the chunk
from the engine's `max_part_mb` (`client.ts:218`, fallback 95), and the engine default is **95
MB** (`origin/master backend/config/models.py:293`, `backend/api/uploads.py:49`). Still under
the 100 MB cap, so it functions; the doc's "?90 MB" is just inaccurate.
- (d) Cross-origin reel download via Content-Disposition ? CONFIRMED: `<a download>` to the
cross-origin `api.` URL (`CompileResult.tsx:68-74`), and the engine serves it with
`FileResponse(..., filename=name)` (`origin/master backend/main.py:690`), which emits `Content-
Disposition: attachment`. Doc §7 line 129 correctly flags the header as authoritative.

Severity minor ? nothing breaks the deploy; the owner should either set `max_part_mb<=90` or
correct the doc to ~95 MB.